STA 336: Statistical Machine Learning

Homework 5

Rento Saijo

March 6, 2026

Table of Contents

Disclosure **2**

Problem 1 **3**
Problem 1 (a) 4
Problem 1 (b) 5

Problem 2 **6**
Problem 2 (a) 7
Problem 2 (b) 8
Problem 2 (c) 9
Problem 2 (d) 11

Problem 3 **13**
Problem 3 (a) 14
Problem 3 (b) 15
Problem 3 (c) 16
Problem 3 (d) 18
Problem 3 (e) 19
Problem 3 (f) 20

Disclosure

GPT-5.3-Codex was used to create the `YAML` portion and some `LaTeX` code to format the text/equations nicely. Page formatting code was also provided by Derin Gezgin. In the setup chunk, libraries were loaded and some helper functions were defined including but not limited to `table_latex()` and `with_family()`. See the original `RMD` file here for more details.

Problem 1

Problem 1

We now review k-fold cross-validation.

Problem 1 (a)

Problem 1 (a)

Explain how k-fold cross-validation is implemented.

k-fold cross-validation is implemented by randomly splitting the data into k roughly equal-sized folds. For each fold $j = 1, \ldots, k$, we fit the model on the other $k - 1$ folds (the training portion) and evaluate prediction error on fold j (the hold-out portion). This gives k validation errors, one from each fold serving as the validation set once. The final cross-validation estimate is the average of these k errors:

$$\mathrm{CV}_{(k)} = \frac{1}{k} \sum_{j=1}^{k} \mathrm{MSE}_j$$

(or the analogous average classification error for a classification problem).

Problem 1 (b)

Problem 1 (b)

What are the advantages and disadvantages of k-fold cross-validation relative to:

i. The validation set approach?
ii. LOOCV?

i. Relative to the validation set approach, k-fold cross-validation usually gives a more stable and reliable estimate of test error because every observation is used for both training and validation across folds. It also reduces dependence on one arbitrary split. The tradeoff is that it is computationally more expensive than fitting a model once on a single train/validation split.

ii. Relative to LOOCV, k-fold cross-validation (for example, $k = 5$ or $k = 10$) is usually much faster because we fit the model k times instead of n times. It can also have lower variance in the error estimate than LOOCV because each training set differs more substantially across folds. The tradeoff is that LOOCV often has slightly lower bias because each training fit uses $n - 1$ observations, while k-fold uses only $(k - 1)n/k$ observations in each fit.

Problem 2

Problem 2

In Chapter 4, we used logistic regression to predict the probability of `default` using `income` and `balance` on the `Default` data set. We will now estimate the test error of this logistic regression model using the validation set approach. Do not forget to set a random seed before beginning your analysis.

Cell 1

```r
# Load data.
default_df <- ISLR2::Default

# Helper for validation-set error.
validation_error <- function(seed, formula, cutoff = 0.5) {
  set.seed(seed)
  n <- nrow(default_df)
  train_idx <- sample(seq_len(n), n / 2)
  train_df <- default_df[train_idx, ]
  valid_df <- default_df[-train_idx, ]
  model <- stats::glm(formula, data = train_df, family = stats::binomial)
  prob <- stats::predict(model, newdata = valid_df, type = 'response')
  pred <- ifelse(prob > cutoff, 'Yes', 'No')
  list(
    seed = seed,
    train_n = nrow(train_df),
    valid_n = nrow(valid_df),
    misclassified = sum(pred != valid_df$default),
    error = mean(pred != valid_df$default),
    model = model
  )
}
```

Problem 2 (a)

Problem 2 (a)

Fit a logistic regression model that uses `income` and `balance` to predict `default`.

Cell 2

```r
# Fit model on full data.
model_2a <- stats::glm(default ~ income + balance, data = default_df, family =
stats::binomial)

# Create coefficient table.
coef_2a_tbl <- as.data.frame(stats::coef(summary(model_2a))) %>%
  tibble::rownames_to_column(var = 'term') %>%
  dplyr::rename(
    estimate = Estimate,
    std_error = `Std. Error`,
    z_value = `z value`,
    p_value = `Pr(>|z|)`
  ) %>%
  dplyr::mutate(
    estimate = signif(estimate, 6),
    std_error = signif(std_error, 6),
    z_value = signif(z_value, 6),
    p_value = signif(p_value, 6)
  )
coef_2a_tbl %>%
  table_latex(
    col_names = c('Term', 'Estimate', 'Std. Error', 'z value', 'Pr(>|z|)'),
    caption = 'Logistic regression fit for default ~ income + balance (full
    data).'
  )
```

Table 1: Logistic regression fit for default income + balance (full data).

Term	Estimate	Std. Error	z value	Pr(>|z|)
(Intercept)	-11.5405000	0.4347560	-26.54470	0.0000000
income	0.0000208	0.0000050	4.17418	0.0000299
balance	0.0056471	0.0002274	24.83630	0.0000000

The fitted model is

$$\log\left(\frac{\hat{p}}{1-\hat{p}}\right) = -11.5405 + 0.000020809\,\texttt{income} + 0.00564710\,\texttt{balance},$$

where $\hat{p} = \Pr(\texttt{default} = \texttt{Yes} \mid \texttt{income}, \texttt{balance})$.

Problem 2 (b)

Problem 2 (b)

Using the validation set approach, estimate the test error of this model. In order to do this, you must perform the following steps:

i. Split the sample set into a training set and a validation set.
ii. Fit a multiple logistic regression model using only the training observations.
iii. Obtain a prediction of default status for each individual in the validation set by computing the posterior probability of default for that individual, and classifying the individual to the `default` category if the posterior probability is greater than 0.5.
iv. Compute the validation set error, which is the fraction of the observations in the validation set that are misclassified.

Cell 3

```r
# Validation estimate with one split.
res_2b <- validation_error(seed = 1, formula = default ~ income + balance, cutoff = 0.5)
res_2b_tbl <- tibble::tibble(
  seed = res_2b$seed,
  training_observations = res_2b$train_n,
  validation_observations = res_2b$valid_n,
  misclassified = res_2b$misclassified,
  validation_error = round(res_2b$error, 4)
)
res_2b_tbl %>%
  table_latex(
    col_names = c('Seed', 'Training n', 'Validation n', 'Misclassified', 'Validation Error'),
    caption = 'Validation-set estimate for default ~ income + balance.'
  )
```

Table 2: Validation-set estimate for default income + balance.

Seed	Training n	Validation n	Misclassified	Validation Error
1	5000	5000	127	0.0254

With seed 1, the validation error is $\boxed{0.0254}$ (127 misclassified out of 5000).

Problem 2 (c)

Problem 2 (c)

Repeat the process in (b) three times, using three different splits of the observations into a training set and a validation set. Comment on the results obtained.

Cell 4

```r
# Repeat with three different seeds/splits.
seeds_2c <- c(2, 3, 4)
errs_2c <- sapply(seeds_2c, function(s) validation_error(seed = s, formula =
default ~ income + balance)$error)
res_2c_tbl <- tibble::tibble(
  seed = seeds_2c,
  validation_error = round(errs_2c, 4)
)
summary_2c_tbl <- tibble::tibble(
  metric = c('Mean error', 'Minimum error', 'Maximum error'),
  value = round(c(mean(errs_2c), min(errs_2c), max(errs_2c)), 4)
)
res_2c_tbl %>%
  table_latex(
    col_names = c('Seed', 'Validation Error'),
    caption = 'Validation-set errors over three different random splits.'
  )
```

Table 3: Validation-set errors over three different random splits.

Seed	Validation Error
2	0.0238
3	0.0264
4	0.0256

Cell 5

```r
# Create table.
summary_2c_tbl %>%
  table_latex(
    col_names = c('Metric', 'Value'),
    caption = 'Summary of validation-set error across the three splits.'
  )
```

Table 4: Summary of validation-set error across the three splits.

Metric	Value
Mean error	0.0253
Minimum error	0.0238
Maximum error	0.0264

The three validation-set errors are 0.0238, 0.0264, and 0.0256. These values are close to one another, so the estimated test error is fairly stable, but not identical across splits because the validation set approach depends on which observations happen to fall in training versus validation.

Problem 2 (d)

Problem 2 (d)

Now consider a logistic regression model that predicts the probability of `default` using `income`, `balance`, and a dummy variable for `student`. Estimate the test error for this model using the validation set approach. Comment on whether or not including a dummy variable for `student` leads to a reduction in the test error rate.

Cell 6

```r
# Compare models on the same split (seed = 1).
res_2d <- validation_error(seed = 1, formula = default ~ income + balance +
student, cutoff = 0.5)
compare_2d_tbl <- tibble::tibble(
  model = c('income + balance', 'income + balance + student'),
  validation_error = round(c(res_2b$error, res_2d$error), 4)
)
diff_2d_tbl <- tibble::tibble(
  metric = 'Error difference (with student - without student)',
  value = round(res_2d$error - res_2b$error, 4)
)
compare_2d_tbl %>%
  table_latex(
    col_names = c('Model', 'Validation Error'),
    caption = 'Validation-set comparison on the same random split (seed = 1).'
  )
```

Table 5: Validation-set comparison on the same random split (seed = 1).

Model	Validation Error
income + balance	0.0254
income + balance + student	0.0260

Cell 7

```r
# Create table.
diff_2d_tbl %>%
  table_latex(
    col_names = c('Metric', 'Value'),
    caption = 'Difference in validation error after adding student.'
  )
```

Table 6: Difference in validation error after adding student.

Metric	Value
Error difference (with student - without student)	0.0006

Using the same split as part (b), the model with `student` has validation error $\boxed{0.0260}$, compared with 0.0254 without `student`. Here, adding `student` does not reduce the estimated test error; it increases it slightly by 0.0006.

Problem 3

Problem 3

We will now perform cross-validation on a simulated data set.

Problem 3 (a)

Problem 3 (a)

Generate a simulated data set as follows:

```
set.seed(1)
x <- rnorm(100)
y <- x - 2 * x^2 + rnorm(100)
```

In this data set, what is n and what is p? Write out the model used to generate the data in equation form.

Cell 8

```r
# Simulate data exactly as requested.
set.seed(1)
x <- rnorm(100)
y <- x - 2 * x^2 + rnorm(100)
sim_df <- data.frame(x = x, y = y)

# Report n and p.
np_3a_tbl <- tibble::tibble(
  quantity = c('n (number of observations)', 'p (number of predictors)'),
  value = c(nrow(sim_df), 1)
)
np_3a_tbl %>%
  table_latex(
    col_names = c('Quantity', 'Value'),
    caption = 'Dimensions of the simulated data set.'
  )
```

Table 7: Dimensions of the simulated data set.

Quantity	Value
n (number of observations)	100
p (number of predictors)	1

For this simulated data set, $n = 100$ and $p = 1$ (the single predictor is X). The data-generating model is

$$Y = X - 2X^2 + \varepsilon, \qquad X \sim N(0,1), \qquad \varepsilon \sim N(0,1).$$

Problem 3 (b)

Problem 3 (b)

Create a scatter plot of X against Y. Comment on what you find.

Cell 9

```r
# Create scatter plot.
plot(
  sim_df$x, sim_df$y,
  xlab = 'X',
  ylab = 'Y',
  pch = 19,
  col = '#1f77b4'
)
```

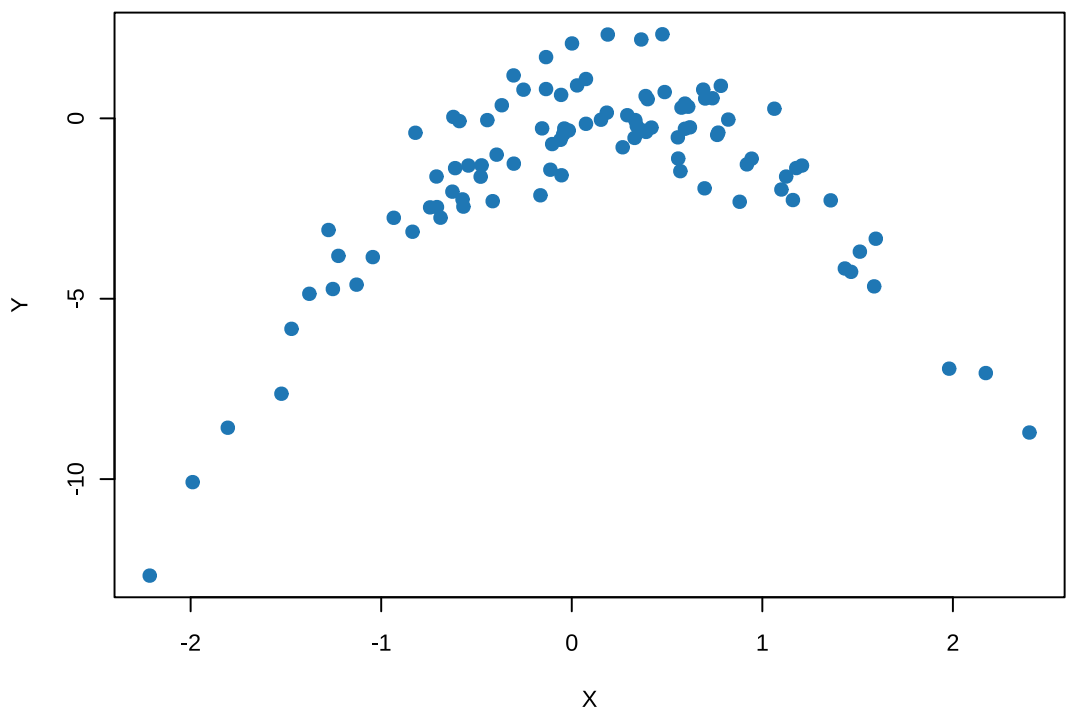

Figure 1: Scatter plot of X against Y for the simulated data.

The scatterplot shows a clear curved (roughly downward-opening parabola) relationship between X and Y, with random noise around that curve. This matches the quadratic signal in the data-generating model.

Problem 3 (c)

Problem 3 (c)

Set a random seed, and then compute the LOOCV errors that result from fitting the following four models using least squares:

i. $Y = \beta_0 + \beta_1 X + \varepsilon$
ii. $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \varepsilon$
iii. $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \varepsilon$
iv. $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \beta_4 X^4 + \varepsilon$

Note you may find it helpful to use the `data.frame()` function to create a single data set containing both X and Y.

Cell 10

```r
# Fit models and compute LOOCV error.
set.seed(123)
glm_3c_1 <- stats::glm(y ~ x, data = sim_df)
glm_3c_2 <- stats::glm(y ~ x + I(x^2), data = sim_df)
glm_3c_3 <- stats::glm(y ~ x + I(x^2) + I(x^3), data = sim_df)
glm_3c_4 <- stats::glm(y ~ x + I(x^2) + I(x^3) + I(x^4), data = sim_df)
loocv_3c <- tibble::tibble(
  model = c('i', 'ii', 'iii', 'iv'),
  specification = c(
    'Y ~ X',
    'Y ~ X + X^2',
    'Y ~ X + X^2 + X^3',
    'Y ~ X + X^2 + X^3 + X^4'
  ),
  loocv_error = round(c(
    boot::cv.glm(sim_df, glm_3c_1)$delta[1],
    boot::cv.glm(sim_df, glm_3c_2)$delta[1],
    boot::cv.glm(sim_df, glm_3c_3)$delta[1],
    boot::cv.glm(sim_df, glm_3c_4)$delta[1]
  ), 6)
)
loocv_3c %>%
  table_latex(
    col_names = c('Model', 'Specification', 'LOOCV Error'),
    caption = 'LOOCV errors for polynomial models (seed = 123).',
    escape = TRUE
  )
```

Table 8: LOOCV errors for polynomial models (seed = 123).

Model	Specification	LOOCV Error
i	Y ~ X	7.288162
ii	Y ~ X + X^2	0.937424
iii	Y ~ X + X^2 + X^3	0.956622
iv	Y ~ X + X^2 + X^3 + X^4	0.953905

With seed 123, the LOOCV errors are:

i: 7.288162, ii: 0.937424, iii: 0.956622, iv: 0.953905.

Problem 3 (d)

Problem 3 (d)

Repeat (c) using another random seed, and report your results. Are your results the same as what you got in (c)? Why?

Cell 11

```r
# Repeat LOOCV with a different seed.
set.seed(999)
glm_3d_1 <- stats::glm(y ~ x, data = sim_df)
glm_3d_2 <- stats::glm(y ~ x + I(x^2), data = sim_df)
glm_3d_3 <- stats::glm(y ~ x + I(x^2) + I(x^3), data = sim_df)
glm_3d_4 <- stats::glm(y ~ x + I(x^2) + I(x^3) + I(x^4), data = sim_df)
loocv_3d <- c(
  boot::cv.glm(sim_df, glm_3d_1)$delta[1],
  boot::cv.glm(sim_df, glm_3d_2)$delta[1],
  boot::cv.glm(sim_df, glm_3d_3)$delta[1],
  boot::cv.glm(sim_df, glm_3d_4)$delta[1]
)
compare_3d_tbl <- tibble::tibble(
  model = c('i', 'ii', 'iii', 'iv'),
  loocv_seed_123 = loocv_3c$loocv_error,
  loocv_seed_999 = round(loocv_3d, 6),
  absolute_difference = round(abs(loocv_3c$loocv_error - loocv_3d), 6)
)
compare_3d_tbl %>%
  table_latex(
    col_names = c('Model', 'LOOCV (Seed 123)', 'LOOCV (Seed 999)', 'Absolute Difference'),
    caption = 'LOOCV errors under two different random seeds.'
  )
```

Table 9: LOOCV errors under two different random seeds.

Model	LOOCV (Seed 123)	LOOCV (Seed 999)	Absolute Difference
i	7.288162	7.288162	0
ii	0.937424	0.937424	0
iii	0.956622	0.956622	0
iv	0.953905	0.953905	0

The LOOCV errors are exactly the same for both seeds. This happens because LOOCV is deterministic for a fixed data set and fixed model: each observation is left out once, so there is no random fold assignment as in k-fold CV with $k < n$.

Problem 3 (e)

Problem 3 (e)

Which of the models in (c) had the smallest LOOCV error? Is this what you expected? Explain your answer.

Cell 12

```r
# Identify best model by LOOCV.
best_idx_3e <- which.min(loocv_3c$loocv_error)
best_3e_tbl <- tibble::tibble(
  model = loocv_3c$model[best_idx_3e],
  specification = loocv_3c$specification[best_idx_3e],
  loocv_error = loocv_3c$loocv_error[best_idx_3e]
)
best_3e_tbl %>%
  table_latex(
    col_names = c('Best Model', 'Specification', 'LOOCV Error'),
    caption = 'Model with smallest LOOCV error.',
    escape = TRUE
  )
```

Table 10: Model with smallest LOOCV error.

Best Model	Specification	LOOCV Error
ii	Y ~ X + X^2	0.937424

Model (ii), the quadratic model $Y \sim X + X^2$, has the smallest LOOCV error (0.937424). Yes, this is what I expected, because the true data-generating process includes a quadratic term ($-2X^2$) and does not include higher-order terms.

Problem 3 (f)

Problem 3 (f)

Comment on the statistical significance of the coefficient estimates that results from fitting each of the models in (c) using least squares. Do these results agree with the conclusions drawn based on the cross-validation results?

Cell 13

```r
# Fit least-squares models.
lm_3f_1 <- stats::lm(y ~ x, data = sim_df)
lm_3f_2 <- stats::lm(y ~ x + I(x^2), data = sim_df)
lm_3f_3 <- stats::lm(y ~ x + I(x^2) + I(x^3), data = sim_df)
lm_3f_4 <- stats::lm(y ~ x + I(x^2) + I(x^3) + I(x^4), data = sim_df)

# Build coefficient significance table.
coef_table <- function(model, model_name) {
  out <- as.data.frame(summary(model)$coefficients) %>%
    tibble::rownames_to_column(var = 'term') %>%
    dplyr::rename(
      estimate = Estimate,
      std_error = `Std. Error`,
      t_value = `t value`,
      p_value = `Pr(>|t|)`
    ) %>%
    dplyr::mutate(model = model_name) %>%
    dplyr::select(model, term, estimate, std_error, t_value, p_value)
  out
}
coef_3f_tbl <- dplyr::bind_rows(
  coef_table(lm_3f_1, 'Model i'),
  coef_table(lm_3f_2, 'Model ii'),
  coef_table(lm_3f_3, 'Model iii'),
  coef_table(lm_3f_4, 'Model iv')
) %>%
  dplyr::mutate(
    estimate = signif(estimate, 6),
    std_error = signif(std_error, 6),
    t_value = signif(t_value, 6),
    p_value = signif(p_value, 6)
  )
coef_3f_tbl %>%
  table_latex(
    col_names = c('Model', 'Term', 'Estimate', 'Std. Error', 't value',
    'Pr(>|t|)'),
    caption = 'Coefficient significance from least-squares fits in models
    (i)-(iv).',
    escape = TRUE
  )
```

Table 11: Coefficient significance from least-squares fits in models (i)-(iv).

Model	Term	Estimate	Std. Error	t value	Pr(>|t|)
Model i	(Intercept)	-1.6254300	0.2619370	-6.205420	0.0000000
Model i	x	0.6924970	0.2909420	2.380190	0.0192385
Model ii	(Intercept)	0.0567150	0.1176550	0.482043	0.6308610
Model ii	x	1.0171600	0.1079830	9.419670	0.0000000
Model ii	I(x^2)	-2.1189200	0.0847657	-24.997400	0.0000000
Model iii	(Intercept)	0.0615072	0.1195040	0.514688	0.6079540
Model iii	x	0.9752800	0.1872810	5.207560	0.0000011
Model iii	I(x^2)	-2.1237900	0.0870025	-24.410700	0.0000000
Model iii	I(x^3)	0.0176386	0.0642904	0.274358	0.7843990
Model iv	(Intercept)	0.1567030	0.1394620	1.123630	0.2640030
Model iv	x	1.0308300	0.1913370	5.387500	0.0000005
Model iv	I(x^2)	-2.4099000	0.2348550	-10.261200	0.0000000
Model iv	I(x^3)	-0.0091329	0.0672288	-0.135848	0.8922290
Model iv	I(x^4)	0.0697854	0.0532401	1.310770	0.1930960

In model (ii), both X and X^2 are highly significant (p-values 2.40×10^{-15} and 4.58×10^{-44}, respectively). In model (iii), X^3 is not significant ($p = 0.7844$). In model (iv), both X^3 and X^4 are not significant ($p = 0.8922$ and $p = 0.1931$). These least-squares significance results agree with the cross-validation conclusion: the quadratic model captures the real signal, while higher-order terms do not improve predictive performance.